# Team 71 – Neureset Design

Our Neureset device relies on the Device, Neureset and Session classes to perform the functionalities outlined in the design specifications and use cases. When implementing the device, we've taken inspiration from the observer and mediator design patterns taught in this course.

The Neureset class serves as control point of the device. While the Device class handles and updates the user interface, the Neureset class handles the functionality of the device outlined in the specification document, while the Sessions class acts as the session handler for each stage of the treatment process.

Treatment is delivered by the device through EEG Sensors (instances of the EEGSensors class). The sensors are overseen by an instance of the Sessions class, which handles the treatment process each step of the way. The Sessions class uses various states to control the actions of the EEG Sensors and to deliver the appropriate treatment for the correct amount of time.

The Sessions class uses a state machine to decide which action to take while handling a session. The states move forward when the session timer has run out. We applied this when implementing our EEG Sensors and how they would be handled during different stages of the treatment process. The Sessions class communicates with the EEGSensors by working through various states related to the stages of treatment. By implementing the classes this way, we have encapsulated our classes to be reused appropriately between treatments, and allowed actions in different states to be written into a single function.

Both the Session Log, and date and time functionalities are handled by the Neureset class. The session logs are communicated to and stored at the Neureset class by the Sessions class once treatment has been completed, and the data is then provided to the user upon request.

The observer design pattern is used in our device in many aspects. The observer design pattern is typically used when there is a one-to-many relationship between objects, The Sessions, Neureset and the Device class communicates with each other when their status has been updated. For example, Sessions will inform Device when entering a new state, so that Device can update the session screen and control treatment lights accordingly. Another example would be updating the battery indicator. When Neureset detects a change in battery level, it informs device about the change. Device will then fetch the new battery level from Neureset and update the battery indicator. By using this design pattern, these classes can sync with each other and get the most up-to-date information from each other, and they don't need to actively check for state updates in other classes, which is more efficient.

Finally, we used a mediator design when implementing how our UI would be updated during the device's use. Each screen state has it's own class, and they control how to update the screens themselves. Device will decide which screen to show at the main screen, and will handle information and function exchanges

between the screens and the system (Neureset). The Device class handles updates and changes to various classes, as well as the progression of treatment throughout the device's use, and communicates these updates to the screen classes so that they can update themselves accordingly.